Assignment Report

The Assignment report submitted in partial fulfilment of the

requirements for

the award of the degree of

The Professional Master Of

Embedded And Cyber-Physical Systems

ECPS 216 - F'24: IOT Systems & Software

Programming Assignment 4

Under the guidance of

Prof. Quoc-Viet Dang

Name of Candidates	UCI Student ID
Sahil Kakadiya	93282494

Introduction

This project involves creating a communication network between three ESP8266 devices and a Raspberry Pi to form a "swarm" that monitors light levels.

Overview

1. **ESP8266 devices**: Each device has a light sensor (photoresistor), and the devices communicate wirelessly with each other. One device, with the highest light sensor reading, will act as the "Master" and send the data to the Raspberry Pi for logging and graph plotting.
2. **Raspberry Pi (RPi)**: It will log the data from the Master ESP8266 and display the results. It has a button to reset the ESP8266 swarm, and it also has four LEDs to show the status. And has a facility of logging and graph plotting
3. **Swarm dynamics**: The system must work dynamically with 1–3 ESP8266 devices. The Raspberry Pi should be able to join after the Master device has been identified.

Part 1 - Raspberry Pi Wi-Fi Setup and Packet Delivery

The System Controller class implements a framework for managing a Raspberry Pi's interaction with external hardware and network communication. Here's a breakdown of its functionality and design:

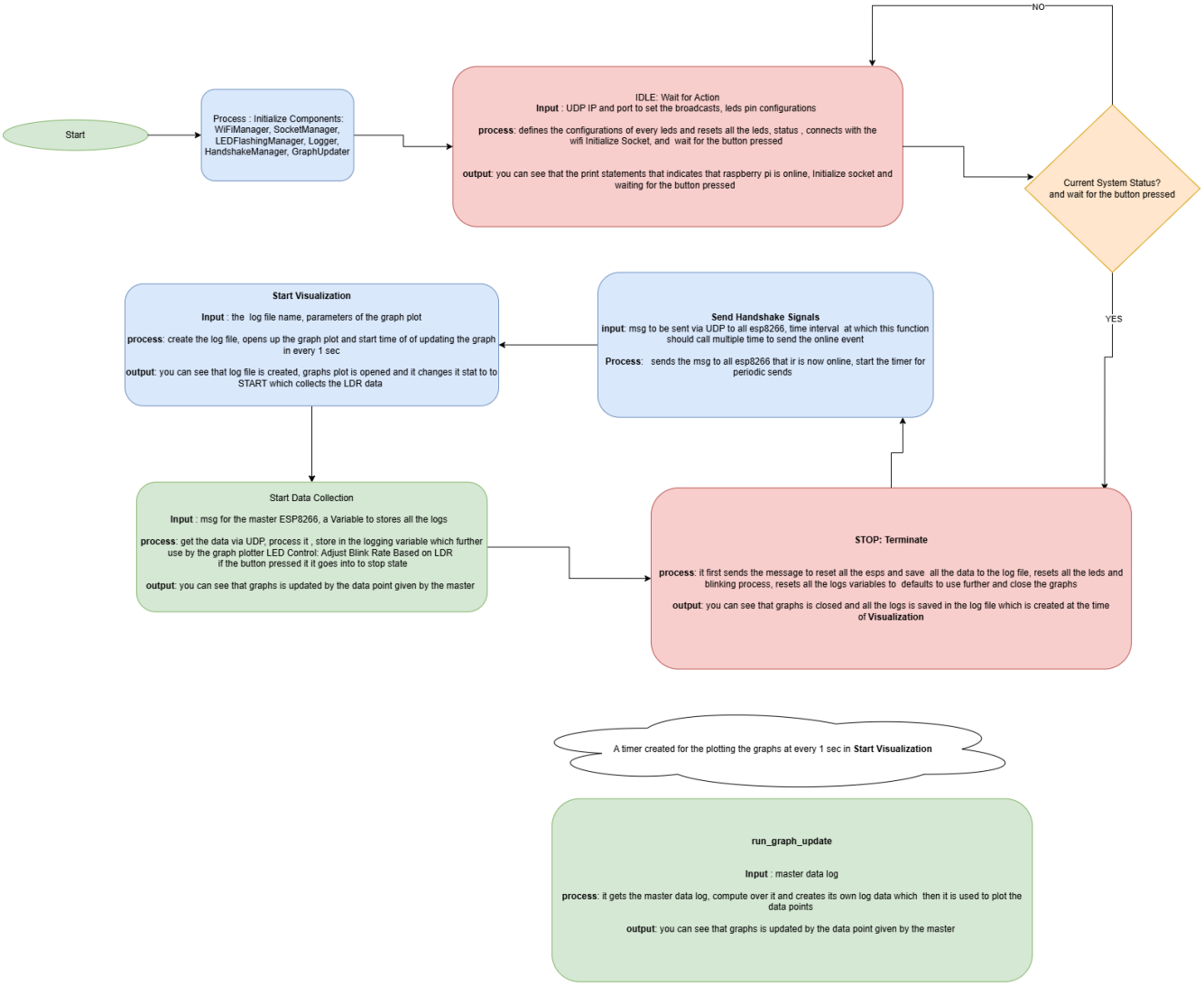
Core Features:

1. **LED Control:**
    - The class manages four LEDs (resetled, rled, gled, bled) and associates each LED with different tasks or master devices via LED_MAPPING_DICT.
2. **Button Interaction:**
    - A button press toggles between different process_stage states (IDLE, START_VISUALIZATION, START_COLLECTING, and STOP).
3. **WiFi Management:**
    - Uses a WiFiManager to ensure the Raspberry Pi stays connected to the network.
    - Automatically reconnects if the WiFi connection drops.
4. **Socket Communication:**
    - The SocketManager handles data transmission.
    - Sensor data from external devices (e.g., ESP8266 boards) is parsed and processed.
5. **Master Device Mapping:**
    - Maps unique IDs of "MASTER" ESP8266 devices to LEDs.
    - Avoids duplicate configurations and ensures each master has a corresponding LED.
6. **Data Logging:**
    - A Logger records events and interactions, which is especially useful for debugging and performance tracking.
7. **Graph Updates:**
    - The GraphUpdater presumably visualizes real-time data such as photocell readings, enhancing monitoring.
8. **Status Management:**
    - Implements a finite state machine (STATUS) to control various system stages.
    - The cleanup method resets the system to a default state, ensuring consistency.

System Overview

1. **Device Management (`MasterManager`)**:
   - Manages and tracks data for master devices.
   - Ensures only one device acts as the master at a time.
   - Updates master data with relevant information such as LDR values, IP, and port.
2. **Wi-Fi Connectivity (`WiFiManager`)**:
   - Checks and attempts to reconnect to Wi-Fi if the connection is lost.
3. **Socket Communication (`SocketManager`)**:
   - Handles initialization, data transmission, and reception using UDP sockets.
   - Allows devices to broadcast and receive messages.
4. **Graph Visualization (`GraphUpdater`)**:
   - Visualizes the cumulative time of master devices using bar charts.
   - Displays LDR sensor readings over time using line graphs.
   - Logs device interactions and maintains a timeline of data for visualization.
5. **LED Indication (`LEDFlashingManager`)**:
   - Controls LEDs for visual feedback.
   - Maps LDR sensor data to blinking rates and updates LEDs accordingly.
   - Resets LEDs when needed.
6. **Handshake Management (`HandshakeManager`)**:
   - Implements UDP-based handshakes for device presence confirmation.
   - Repeatedly sends "online" indicators and handles "reset" signals.
7. **Logging (`Logger`)**:
   - Logs master data and generates summaries.
   - Provides a persistent record of device activities and interactions.

Graphical Flowchart for Raspberry Pi Program:



Part 2 - ESP8266 Wi-Fi Setup and Packet Delivery

This sets up a small network of ESP8266 devices with a Raspberry Pi server. The ESP8266 devices, equipped with light sensors (photoresistors), broadcast their readings over Wi-Fi. They designate a "master" device based on light intensity, which communicates data to a Raspberry Pi. Let's go through the code step-by-step.

1. Wi-Fi and Device Setup

- The program connects the ESP8266 to a predefined Wi-Fi network.
- The ESP8266's IP address and a unique identifier (derived from its MAC address) are assigned.
- Device roles are tracked using an enum (DeviceStatus) for states like IDLE, ACTIVE, and MASTER.

2. UDP Communication

- The code uses UDP for broadcasting packets over the network, initializing communication on a common broadcast IP (255.255.255.255) and port (54321).
- Multiple ESP8266 devices and a Raspberry Pi communicate with each other via UDP messages.
- WiFiUDP handles packet reading and writing functions for data exchange.

3. Device Configuration and Status Tracking

- Each ESP8266 maintains a configuration structure (DeviceConfig) that includes:
    - A unique identifier (UniqueID), IP address, ports for listening/sending, and the light sensor reading (LDR_reading).
    - The master device status (isMaster) is set based on LDR readings.
- The Raspberry Pi configuration is tracked separately using ServerConfig.
- When the Raspberry Pi sends a "RESET" command, devices are reset and de-configured.

4. Broadcasting and Incoming Data Handling

- broadcastData: Regularly broadcasts the device's LDR reading across the network.
- processIncomingPackets: Processes incoming packets to determine if they are from another ESP8266 or from the Raspberry Pi.
- handleIncomingPacket: Handles each packet based on its source:
    - If the packet is from the Raspberry Pi, it is passed to RPI_IncomingPackets.
    - If the packet is from another ESP8266, it is handled by ESP8266_IncomingPackets.

5. Checking and Assigning Master Status

- **Master Selection**: checkandsetMaster scans through configured devices to find the one with the highest LDR reading, assigning it as the "master."
- **Indication**: The master device lights up the master LED (LED_MASTER_PIN) to visually indicate its status.

6. Data Transmission to Raspberry Pi

- The master device sends its LDR reading to the Raspberry Pi through sendToRPi.
- The format of the transmitted message includes the device token, master status, unique ID, and LDR reading.

7. Sensor Reading and Light Intensity Mapping

- **LDR Reading**: readLDR reads the light intensity and updates LDR_reading. This reading determines the device's "master" status if it has the highest reading.
- **LED Intensity**: intensityIndication maps the LDR reading to a flashing frequency for the LED, simulating changes in light intensity by adjusting the blink rate of the LED.

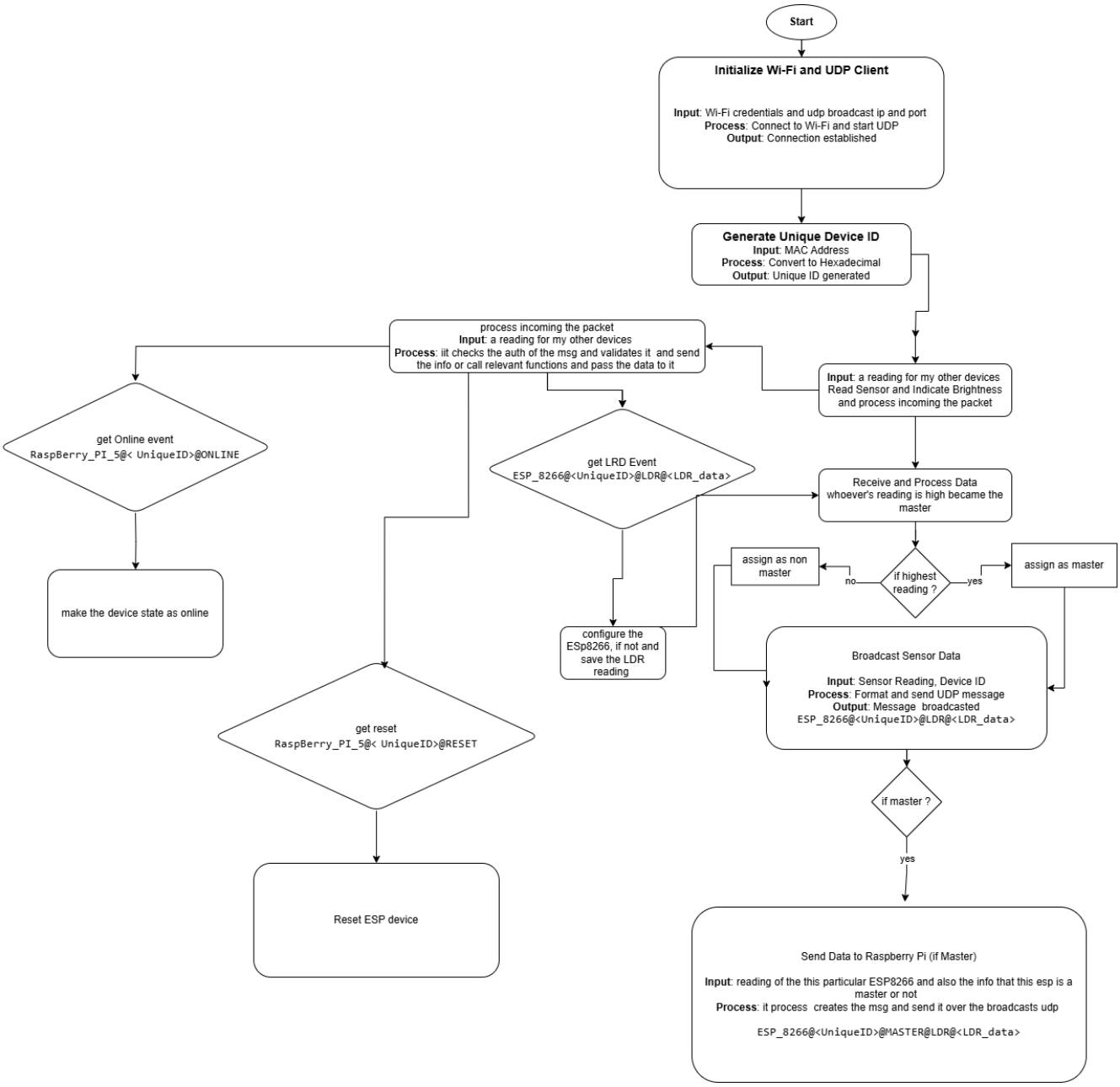
8. Device Status Maintenance

- The ESP8266 devices check for inactivity in other devices and de-configure them if they miss periodic online notifications (OnlineNotification).
- CheckOnlineCountDown reduces the countdown timer for each device; if the countdown reaches zero, the device is de-configured.
- This mechanism prevents inactive devices from occupying slots in the network.

9. Summary of Key Functions

- **Setup Functions**:
  - setup: Initializes Wi-Fi, assigns device IDs, and configures UDP communication.
- **Loop Functions**:
  - loop: Repeatedly processes incoming packets, reads the LDR, and maintains device configuration.
- **Configuration and Broadcast**:
  - device_deConfigure: Resets device configuration.
  - broadcastData and OnlineNotification: Ensure devices are regularly broadcasting their status and data.

Graphical Flowchart for ESP8266 Program:



Part 3- Packet Transfer details

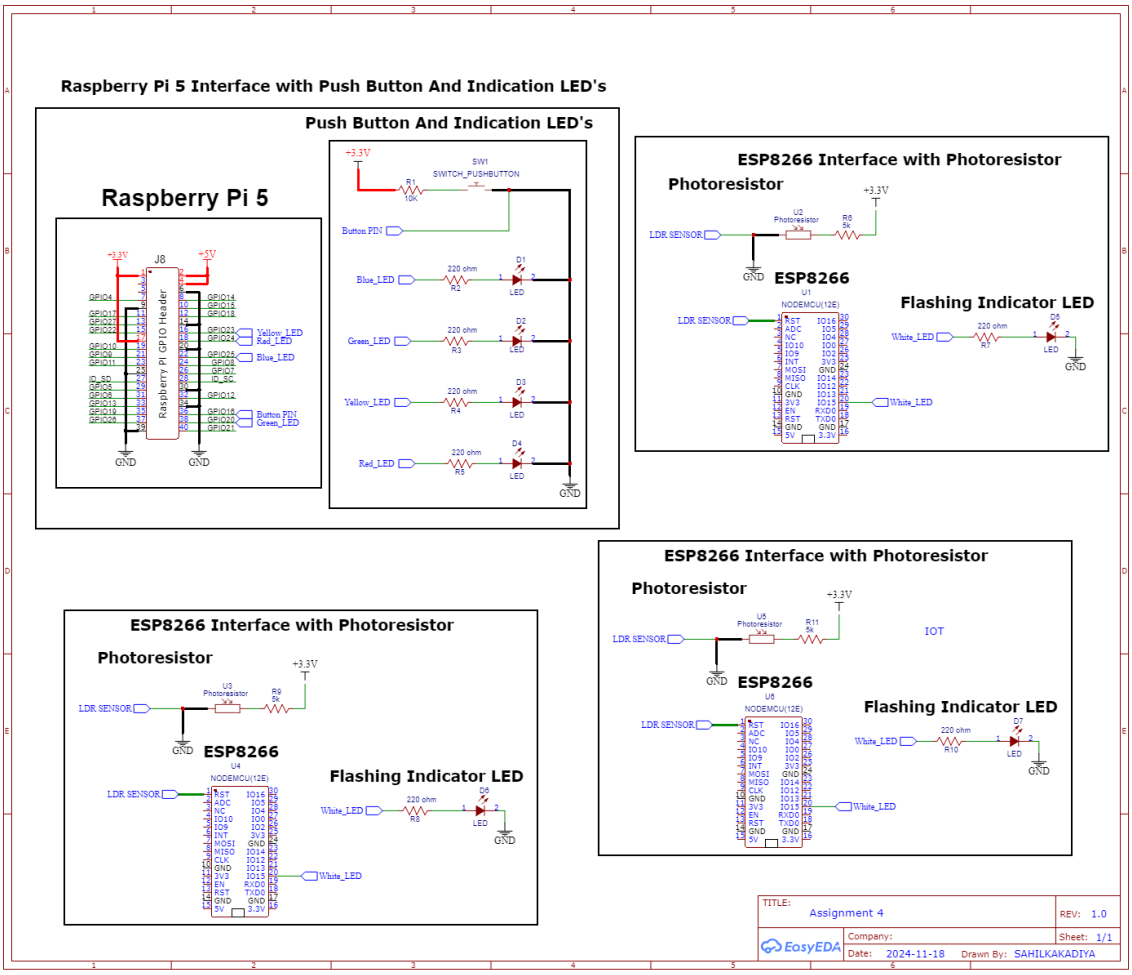
Packet Information:

- **ESP8266 to other Esps and Raspberry Pi**:
  - Online notification: "ESP_8266@<UniqueID>@ONLINE"
  - Master reading: "ESP_8266@<UniqueID>@MASTER@LDR@<LDR_reading>"
  - Broadcast message (e.g., LDR data): "ESP_8266@<UniqueID>@LDR@<LDR_reading>"
- **Raspberry Pi to ESP8266:**
  - Online notification: "RaspBerry_PI_5@<UniqueID>@ONLINE"
  - Reset command: "RaspBerry_PI_5@<UniqueID>@RESET"

Packet Flow and Sequence:

1. **ESP8266 Device Broadcasts LDR Data:**
   - **Packet:**
     `ESP_8266@<UniqueID>@LDR@<LDR_data>`
     Example: `ESP_8266@ESP_1234@LDR@512`
   - **Recipient:** other ESP8266 devices
2. **ESP8266 Device Broadcasts ONLINE Message to All Devices:**
   - **Packet:**
     `ESP_8266@<UniqueID>@ ONLINE`
     Example: `ESP_8266@ESP_1234@ ONLINE`
   - **Recipient:** Raspberry Pi (RPi) or other ESP8266 devices
3. **Raspberry Pi Sends "ONLINE" Message to All ESP8266 Devices:**
   - **Packet:**
     `RaspBerry_PI_5@< UniqueID>@ONLINE`
     Example: `RaspBerry_PI_5@RPi_1@ONLINE`
   - **Recipient:** All ESP8266 devices
4. **ESP8266 Devices Compare LDR Values:**
   - Each ESP8266 device compares its LDR reading with others. The device with the **highest reading** becomes the **Master**.
5. **Master Sends Master Data to Raspberry Pi:**
   - If the device is the **Master**, it sends the following message to the Raspberry Pi:
     Packet:
     `ESP_8266@<UniqueID>@MASTER@LDR@<LDR_data>`
     Example: `ESP_8266@ESP_1234@MASTER@LDR@512`
   - **Recipient:** Raspberry Pi (RPi)
6. **Raspberry Pi Sends "RESET" Command to All ESP8266 Devices:**
   - **Packet:**
     `RaspBerry_PI_5@< UniqueID>@RESET`
     Example: `RaspBerry_PI_5@RPi_1@RESET`
   - **Recipient:** All ESP8266 devices

Part 4 – Schematic



Explanation of the Interface Between Raspberry Pi 5, 3X ESP8266 and External Components (Push Buttons, LEDs, LDR Sensor)

1. Raspberry Pi 5 Interface with Push Button and LED Indicators

- **GPIO Pins:**
  - The **General-Purpose Input/Output (GPIO)** pins on the **Raspberry Pi 5** are used to interface with external components like push buttons and LED indicators.

- The Raspberry Pi's GPIO header (J8) has multiple pins, including **3.3V**, **5V**, **GND**, and several programmable input/output pins. These GPIO pins allow the Raspberry Pi to receive signals (e.g., from a push button) or send signals (e.g., to LEDs to turn them on or off).

- **Push Button (SW1):**
  - A **push button** is connected to one of the GPIO pins of the Raspberry Pi.
  - **One side of the push button** is connected to the GPIO pin via a **10kΩ resistor (R1)**, acting as a pull-down resistor. This ensures that when the button is not pressed, the GPIO pin reads a **LOW** signal (0V).
  - **The other side of the button** is connected to **3.3V**. When the button is pressed, the GPIO pin is pulled high (3.3V), and the Raspberry Pi can detect this state change as a **button press**.
- **LED Indicators:**
  - Four **LEDs** are connected to separate GPIO pins, allowing the Raspberry Pi to control each LED individually by sending signals through these pins.
  - Each LED has a **220ohm current-limiting resistor** connected in series with it to protect the LED and the Raspberry Pi's GPIO pins from excessive current.
  - The following LEDs are connected:
    1. **Yellow LED** via resistor **R2**
    2. **Green LED** via resistor **R3**
    3. **blue LED** via resistor **R4**
    4. **Red LED** via resistor **R5**
  - The Raspberry Pi can turn these LEDs on or off by toggling the GPIO pin to **HIGH** (3.3V) or **LOW** (0V).
- **Power and Ground:**
  - The **3.3V** and **5V** pins on the Raspberry Pi provide power to components that need it, like sensors or circuits.
  - All components (push button and LEDs) are grounded via the **GND pin** of the Raspberry Pi, ensuring a complete electrical circuit for proper functioning.

---

2. 3X ESP8266 Interface with LDR Sensor and external led indicator

- **LDR (Photoresistor) Sensor:**
  - The **Light Dependent Resistor (LDR)** is used to measure light intensity. Its resistance varies based on the amount of light falling on it — the more light, the lower its resistance, and vice versa.

- This change in resistance is used to create a **voltage divider circuit** with a fixed resistor.

- **Voltage Divider Circuit:**
  - **One side of the LDR** is connected to **+3.3V**, which provides power to the sensor.
  - **The other side of the LDR** is connected to **GND** through a **5kΩ resistor (R6)**. This forms a voltage divider, which means that the voltage at the point between the LDR and the resistor changes depending on the light intensity sensed by the LDR.
  - As the LDR's resistance changes with light intensity, the voltage at the point between the LDR and the resistor also changes. This voltage is proportional to the light intensity.
- **ADC Pin on ESP8266:**
  - The **Analog-to-Digital Converter (ADC)** pin of the **ESP8266** (Pin **A0**) is connected to the point between the LDR and the 5kΩ resistor. The ADC pin reads the varying voltage from the voltage divider circuit.
  - The ESP8266 has only **one ADC pin (A0)**, which can measure voltages between 0V and 1V. Any higher voltage needs to be scaled down appropriately (usually done with resistors or a voltage divider).
  - The ESP8266 uses the analog voltage signal from the LDR to interpret the current light level. This information can then be processed, transmitted over Wi-Fi, or used to control other devices.

- **White LED Indicator:**
  - The white led indicator is used to indicates the ldr data in terms of flashing rates.